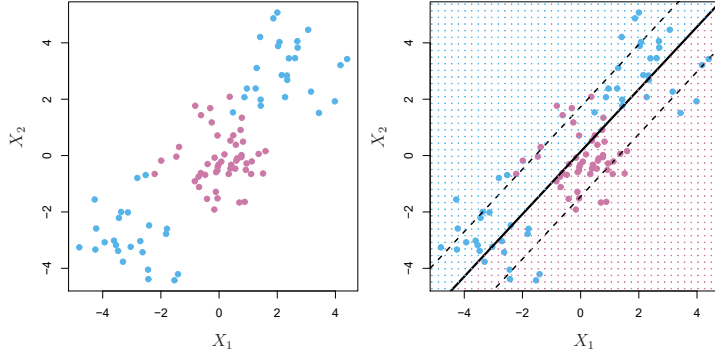


9.3 Support Vector Machines

Trước hết, ta thảo luận một cơ chế tổng quát để chuyển một linear classifier thành classifier tạo ra non-linear decision boundaries. Sau đó ta giới thiệu *support vector machine*, phương pháp thực hiện việc này một cách tự động.



HÌNH 9.8. Left: Các observations thuộc two classes, với một non-linear boundary giữa chúng. Right: Support vector classifier tìm một linear boundary và vì thế perform rất kém.

9.3.1 Classification with Non-Linear Decision Boundaries

Support vector classifier là một approach tự nhiên cho classification trong two-class setting nếu boundary giữa two classes là linear. Tuy nhiên trong practice, ta đôi khi gặp non-linear class boundaries. Ví dụ, xét data ở left-hand panel của Figure 9.8. Rõ ràng support vector classifier, hay bất kỳ linear classifier nào, sẽ perform kém ở đây. Thật vậy, support vector classifier ở right-hand panel của Figure 9.8 gần như vô dụng.

Trong Chapter 7 ta gặp một situation tương tự. Ở đó, performance của linear regression có thể giảm khi relationship giữa predictors và outcome là non-linear. Khi ấy, ta mở rộng feature space bằng các functions của predictors, chẳng hạn quadratic và cubic terms, để xử lý non-linearity. Với support vector classifier, ta cũng có thể xử lý khả năng class boundary là non-linear bằng cách mở rộng feature space bằng quadratic, cubic, và thậm chí higher-order polynomial functions của predictors. Chẳng hạn, thay vì fit support vector classifier với p features

$$X_1, X_2, \dots, X_p,$$

ta có thể fit với $2p$ features

$$X_1, X_1^2, X_2, X_2^2, \dots, X_p, X_p^2.$$

Khi đó, (9.12)–(9.15) trở thành

$$\begin{aligned}
 & \underset{\beta_0, \beta_{11}, \beta_{12}, \dots, \beta_{p1}, \beta_{p2}, \varepsilon_1, \dots, \varepsilon_n, M}{\text{maximize}} && M \\
 & \text{subject to} && y_i \left(\beta_0 + \sum_{j=1}^p \beta_{j1} x_{ij} + \sum_{j=1}^p \beta_{j2} x_{ij}^2 \right) \geq M(1 - \varepsilon_i), \\
 & && \sum_{i=1}^n \varepsilon_i \leq C, \quad \varepsilon_i \geq 0, \quad \sum_{j=1}^p \sum_{k=1}^2 \beta_{jk}^2 = 1.
 \end{aligned} \tag{9.16}$$

Vì sao cách này dẫn đến non-linear decision boundary? Trong enlarged feature space, decision boundary từ (9.16) thực ra là linear. Nhưng trong original feature space, decision boundary có dạng $q(x) = 0$, trong đó q là quadratic polynomial, và các solutions của nó nói chung là non-linear. Ta còn có thể mở rộng feature space bằng higher-order polynomial terms hoặc interaction terms dạng $X_j X_{j'}$ với $j \neq j'$. Hoặc có thể xét các functions khác của predictors thay cho polynomials. Có rất nhiều cách để enlarge feature space; nếu không cẩn thận, number of features có thể trở nên khổng lồ và computations trở nên không khả thi. Support vector machine, được trình bày tiếp theo, cho phép enlarge feature space của support vector classifier theo cách dẫn đến computations hiệu quả.

9.3.2 The Support Vector Machine

Support vector machine (SVM) là extension của support vector classifier, tạo ra bằng cách enlarge feature space theo một cách cụ thể, sử dụng *kernels*. Details của extension này khá complex và nằm ngoài scope của sách; tuy nhiên main idea đã xuất hiện ở Section 9.3.1: ta muốn enlarge feature space để accommodate một non-linear boundary giữa classes. Kernel approach đơn giản là một computationally efficient way để thực hiện ý tưởng đó.

*support vector
machine; kernel*

Ta chưa thảo luận chính xác cách compute support vector classifier vì details hơi technical. Tuy nhiên, solution của problem (9.12)–(9.15) chỉ involve *inner products* của observations, thay vì observations themselves. Inner product của hai r -vectors a và b được định nghĩa là $\langle a, b \rangle = \sum_{i=1}^r a_i b_i$. Do đó inner product của two observations $x_i, x_{i'}$ là

$$\langle x_i, x_{i'} \rangle = \sum_{j=1}^p x_{ij} x_{i'j}. \tag{9.17}$$

Có thể chứng minh rằng:

- Linear support vector classifier có thể được biểu diễn dưới dạng

$$f(x) = \beta_0 + \sum_{i=1}^n \alpha_i \langle x, x_i \rangle, \tag{9.18}$$

trong đó có n parameters α_i , $i = 1, \dots, n$, một parameter cho mỗi training observation.

- Để estimate $\alpha_1, \dots, \alpha_n$ và β_0 , ta chỉ cần $\binom{n}{2}$ inner products $\langle x_i, x_{i'} \rangle$ giữa mọi pairs of training observations. Ký hiệu $\binom{n}{2} = n(n-1)/2$ là number of pairs trong một set gồm n items.

Trong (9.18), để evaluate $f(x)$ ta cần compute inner product giữa new point x và từng training point x_i . Tuy nhiên α_i chỉ nonzero đối với support vectors trong solution: nếu một training observation không phải support vector thì $\alpha_i = 0$. Nếu $\mathcal{S}$ là collection of indices của support points, ta có thể rewrite (9.18) thành

$$f(x) = \beta_0 + \sum_{i \in \mathcal{S}} \alpha_i \langle x, x_i \rangle, \quad (9.19)$$

thường có ít terms hơn nhiều so với (9.18).²

Tóm lại, để represent linear classifier $f(x)$ và compute coefficients của nó, tất cả những gì ta cần là inner products.

Bây giờ, giả sử mỗi khi inner product (9.17) xuất hiện trong representation (9.18), hoặc trong calculation của solution cho support vector classifier, ta thay nó bằng một generalization của inner product dạng

$$K(x_i, x_{i'}), \quad (9.20)$$

trong đó K là một function gọi là *kernel*. Kernel là function định lượng similarity giữa two observations.

Ví dụ, ta có thể đơn giản chọn

$$K(x_i, x_{i'}) = \sum_{j=1}^p x_{ij} x_{i'j}, \quad (9.21)$$

điều này đưa ta trở lại support vector classifier. Equation 9.21 gọi là *linear kernel* vì support vector classifier linear theo features; linear kernel về cơ bản quantifies similarity của a pair of observations bằng Pearson (standard) correlation.

linear kernel

Ta cũng có thể chọn một form khác cho (9.20). Chẳng hạn thay mỗi $\sum_{j=1}^p x_{ij} x_{i'j}$ bằng

$$K(x_i, x_{i'}) = \left(1 + \sum_{j=1}^p x_{ij} x_{i'j} \right)^d. \quad (9.22)$$

Đây là *polynomial kernel* degree d , với d là positive integer. Dùng kernel này với $d > 1$ thay cho standard linear kernel (9.21) trong support vector classifier algorithm tạo ra decision boundary flexible hơn nhiều. Về bản chất, điều này tương đương fit support vector classifier trong higher-dimensional space chứa degree- d polynomials thay vì original feature space.

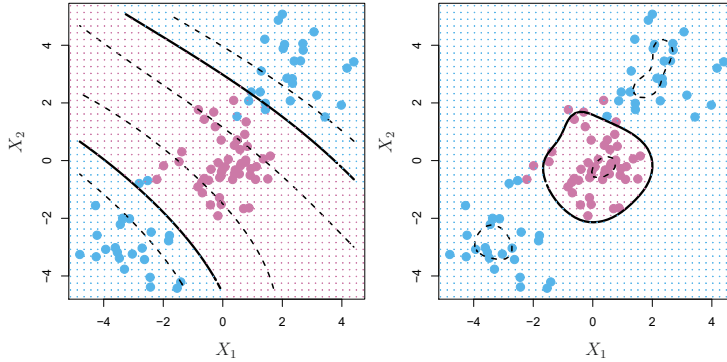
polynomial kernel

Khi support vector classifier được combine với một non-linear kernel như (9.22), resulting classifier gọi là support vector machine. Khi đó (non-linear) function có dạng

$$f(x) = \beta_0 + \sum_{i \in \mathcal{S}} \alpha_i K(x, x_i). \quad (9.23)$$

Left-hand panel của Figure 9.9 cho example SVM với polynomial kernel trên non-linear data của Figure 9.8; fit cải thiện rõ rệt so với linear support

²Bằng cách expand từng inner product trong (9.19), dễ thấy $f(x)$ là linear function của coordinates của x . Việc này cũng thiết lập correspondence giữa α_i và original parameters β_j .



HÌNH 9.9. Left: SVM với polynomial kernel degree 3 được apply lên non-linear data từ Figure 9.8, cho decision rule phù hợp hơn nhiều. Right: SVM với radial kernel được apply. Trong example này, cả hai kernels đều capture được decision boundary.

vector classifier. Khi $d = 1$, SVM reduce về support vector classifier đã thấy trước đó.

Polynomial kernel trong (9.22) chỉ là một example; còn nhiều non-linear kernels khác. Một popular choice là *radial kernel*, có dạng

$$K(x_i, x_{i'}) = \exp \left(-\gamma \sum_{j=1}^p (x_{ij} - x_{i'j})^2 \right), \quad (9.24)$$

trong đó γ là positive constant. Right-hand panel của Figure 9.9 cho example SVM với radial kernel trên data này; nó cũng separate two classes tốt.

radial kernel

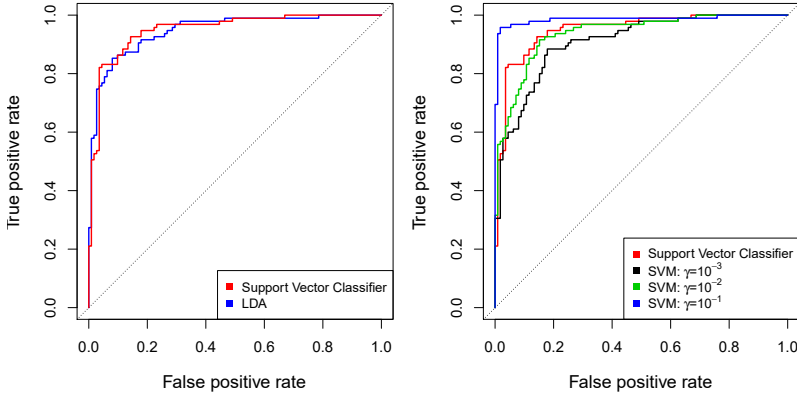
Radial kernel (9.24) hoạt động thế nào? Nếu test observation $x^* = (x_1^*, \dots, x_p^*)^T$ ở xa training observation x_i theo Euclidean distance, thì $\sum_j (x_j^* - x_{ij})^2$ large và $K(x^*, x_i)$ tiny. Vì thế trong (9.23), x_i hầu như không đóng vai trò trong $f(x^*)$. Predicted class label của x^* dựa trên sign của $f(x^*)$, nên training observations far from x^* essentially không ảnh hưởng class label. Radial kernel do đó có behavior rất local: chỉ nearby training observations có ảnh hưởng đáng kể.

Advantage của kernel so với explicit enlargement của feature space như (9.16) là computational. Với kernels, chỉ cần compute $K(x_i, x_{i'})$ cho tất cả $\binom{n}{2}$ distinct pairs, mà không phải explicitly work trong enlarged feature space. Điều này important vì trong nhiều SVM applications, enlarged feature space quá large để calculations khả thi. Với một số kernels như radial kernel (9.24), feature space implicit và infinite-dimensional, nên explicit computation trong đó thậm chí không thể thực hiện.

9.3.3 An Application to the Heart Disease Data

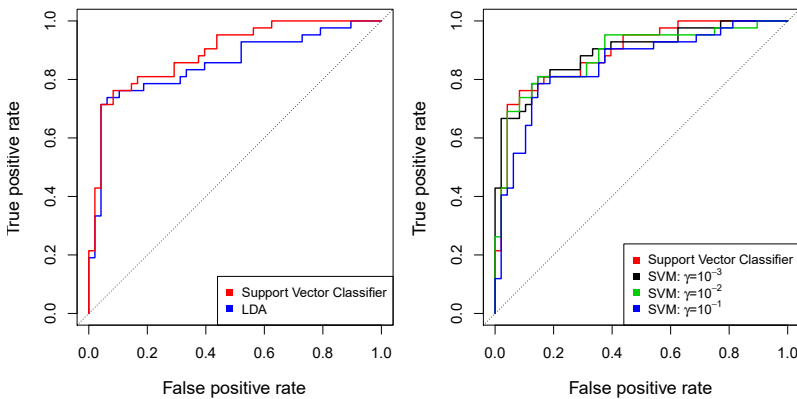
Trong Chapter 8 ta apply decision trees và related methods cho data **Heart**. Goal là dùng 13 predictors như **Age**, **Sex**, và **Chol** để predict một individual có heart disease hay không. Ta bây giờ investigate SVM so với LDA trên data này. Sau khi remove 6 missing observations, data gồm 297 subjects,

random split thành 207 training và 90 test observations.



HÌNH 9.10. ROC curves cho training set của data *Heart*. Left: Support vector classifier và LDA được compare. Right: Support vector classifier được compare với SVM dùng radial basis kernel với $\gamma = 10^{-3}, 10^{-2}, 10^{-1}$.

Trước hết ta fit LDA và support vector classifier lên training data. Support vector classifier tương đương SVM dùng polynomial kernel degree $d = 1$. Left-hand panel của Figure 9.10 display ROC curves (Section 4.4.2) cho training-set predictions của cả LDA và support vector classifier. Cả hai classifiers compute scores dạng $\hat{f}(X) = \hat{\beta}_0 + \hat{\beta}_1 X_1 + \dots + \hat{\beta}_p X_p$. Với cutoff t , observations được classify thành heart disease hoặc no heart disease tùy $\hat{f}(X) < t$ hay $\hat{f}(X) \geq t$. ROC curve được tạo bằng cách form predictions và compute false positive/true positive rates cho range of t . Optimal classifier sẽ hug top-left corner của ROC plot. Trong instance này cả LDA và support vector classifier perform tốt, với suggestion rằng support vector classifier có thể slightly superior.



HÌNH 9.11. ROC curves cho test set của data *Heart*. Left: Support vector classifier và LDA được compare. Right: Support vector classifier được compare với SVM dùng radial basis kernel với $\gamma = 10^{-3}, 10^{-2}, 10^{-1}$.

Right-hand panel của Figure 9.10 display ROC curves cho SVMs với

radial kernel và several values của γ . Khi γ tăng và fit trở nên more non-linear, ROC curves improve; $\gamma = 10^{-1}$ gần như perfect trên training data. Nhưng training error rates có thể misleading về performance trên new test data. Figure 9.11 cho ROC curves trên 90 test observations. Left panel cho support vector classifier có small advantage over LDA, dù differences không statistically significant. Right panel cho thấy SVM với $\gamma = 10^{-1}$, tốt nhất trên training data, lại cho estimates tệ nhất trên test data. Đây là một ví dụ nữa rằng method flexible hơn có thể lower training error nhưng không necessarily improve test performance. SVMs với $\gamma = 10^{-2}$ và 10^{-3} perform tương đương support vector classifier, và cả ba outperform SVM với $\gamma = 10^{-1}$.

9.4 SVMs with More than Two Classes

Cho đến nay discussion chỉ xét binary classification, tức two-class setting. Để extend SVMs tới arbitrary number of classes, separating-hyperplane concept không mở rộng một cách hoàn toàn natural. Có nhiều proposals cho K -class case; hai popular approaches nhất là *one-versus-one* và *one-versus-all*.

9.4.1 One-Versus-One Classification

Giả sử có $K > 2$ classes. One-versus-one, hay all-pairs approach, constructs $\binom{K}{2}$ SVMs, mỗi SVM compares một pair of classes. Ví dụ một SVM compare k th class coded +1 với k' th class coded -1. Ta classify test observation bằng từng classifier và tally number of times observation được assign vào mỗi trong K classes. Final classification assign observation vào class mà nó được assign most frequently trong $\binom{K}{2}$ pairwise classifications.

one-versus-one

9.4.2 One-Versus-All Classification

One-versus-all, còn gọi one-versus-rest, là alternative cho $K > 2$. Ta fit K SVMs, mỗi lần compare một trong K classes với remaining $K - 1$ classes. Gọi $\beta_{0k}, \dots, \beta_{pk}$ là parameters từ SVM comparing k th class (coded +1) với others (coded -1), và x^* là test observation. Ta assign observation vào class có $\beta_{0k} + \beta_{1k}x_1^* + \dots + \beta_{pk}x_p^*$ lớn nhất, vì điều này biểu thị confidence cao rằng observation thuộc k th class hơn các classes khác.

*one-versus-all;
one-versus-rest*

9.5 Relationship to Logistic Regression

Khi SVMs được introduce vào mid-1990s, chúng gây chú ý mạnh trong statistical và machine learning communities. Một phần do good performance và marketing, một phần vì underlying approach dường như vừa novel vừa mysterious. Ý tưởng tìm hyperplane separate data tốt nhất trong khi allow một số violations trông rất khác classical classification approaches như logistic regression và linear discriminant analysis. Việc dùng kernel để expand feature space nhằm accommodate non-linear class boundaries cũng có vẻ là unique, valuable characteristic.

Tuy nhiên, sau đó người ta nhận ra deep connections giữa SVMs và các

classical statistical methods. Criterion (9.12)–(9.15) cho support vector classifier $f(X) = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p$ có thể rewrite thành

$$\underset{\beta_0, \beta_1, \dots, \beta_p}{\text{minimize}} \left\{ \sum_{i=1}^n \max[0, 1 - y_i f(x_i)] + \lambda \sum_{j=1}^p \beta_j^2 \right\}, \quad (9.25)$$

trong đó λ là nonnegative tuning parameter. Khi λ large, $\beta_1, \dots, \beta_p$ small, nhiều margin violations được tolerated và classifier low-variance/high-bias. Khi λ small, ít margin violations occur, tương ứng high-variance/low-bias classifier. Vì vậy small λ trong (9.25) corresponds to small C trong (9.15). Term $\lambda \sum_j \beta_j^2$ là ridge penalty từ Section 6.2.1 và đóng vai trò tương tự trong bias-variance trade-off.

Equation (9.25) có dạng “Loss + Penalty” đã gặp nhiều lần:

$$\underset{\beta_0, \beta_1, \dots, \beta_p}{\text{minimize}} \{L(X, y, \beta) + \lambda P(\beta)\}. \quad (9.26)$$

Ở đây $L(X, y, \beta)$ là loss function quantifying mức độ model parametrized bởi β fits data (X, y) , còn $P(\beta)$ là penalty function trên parameter vector, controlled bởi nonnegative tuning parameter λ .

Ví dụ ridge regression và lasso đều có form này với

$$L(X, y, \beta) = \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j \right)^2,$$

và $P(\beta) = \sum_j \beta_j^2$ cho ridge regression, $P(\beta) = \sum_j |\beta_j|$ cho lasso. Trong (9.25), loss function thay vào là

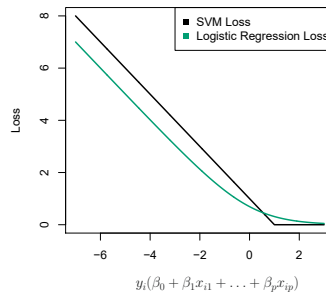
$$L(X, y, \beta) = \sum_{i=1}^n \max[0, 1 - y_i(\beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip})].$$

Loss này gọi là *hinge loss*, được depict trong Figure 9.12, và closely related với loss function dùng trong logistic regression.

Một characteristic thú vị của support vector classifier là chỉ support vectors đóng vai trò trong fitted classifier; observations ở correct side của margin không ảnh hưởng. Lý do là loss trong Figure 9.12 exactly zero nếu $y_i(\beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip}) \geq 1$, tương ứng observations ở correct side of margin.³ Ngược lại, logistic-regression loss không exactly zero ở đâu, nhưng rất small cho observations far from decision boundary. Vì loss functions similar, logistic regression và support vector classifier thường cho very similar results. Khi classes well separated, SVMs có xu hướng behave better; trong more overlapping regimes, logistic regression thường được preferred.

Khi support vector classifier và SVM mới được introduce, tuning parameter C trong (9.15) từng bị xem là unimportant “nuisance” parameter có thể set default, ví dụ 1. Nhưng “Loss + Penalty” formulation (9.25) cho thấy không phải vậy. Choice của tuning parameter rất important và determine mức underfit/overfit của model, như Figure 9.7 minh họa.

³Trong hinge-loss + penalty representation này, margin corresponds to value one, và width của margin được determine bởi $\sqrt{\sum_j \beta_j^2}$.



HÌNH 9.12. SVM và logistic regression loss functions được compare theo $y_i(\beta_0 + \beta_1x_{i1} + \dots + \beta_px_{ip})$. Khi quantity này lớn hơn 1, SVM loss bằng zero vì observation ở correct side of margin. Overall, two loss functions có behavior khá similar.

Ta đã thấy support vector classifier closely related to logistic regression và các preexisting statistical methods. SVM có unique ở việc dùng kernels để enlarge feature space cho non-linear class boundaries không? Câu trả lời là không. Ta cũng có thể perform logistic regression hoặc nhiều classification methods khác bằng non-linear kernels; điều này closely related với các non-linear approaches ở Chapter 7. Tuy nhiên vì historical reasons, non-linear kernels widespread hơn nhiều trong context của SVMs.

Dù chưa address ở đây, có một extension của SVM cho regression (quantitative response), gọi là *support vector regression*. Least squares regression seek coefficients sao cho sum of squared residuals nhỏ nhất. Support vector regression thay vào đó minimizes một different loss, trong đó chỉ residuals có absolute value lớn hơn một positive constant mới contribute. Đây là extension của margin concept từ support vector classifiers sang regression setting.

*support vector
regression*

9.6 Lab: Support Vector Machines

Trong lab này, ta dùng library `sklearn.svm` để demonstrate support vector classifier và support vector machine. Trước hết import một số usual libraries.

```
In [1]: import numpy as np
from matplotlib.pyplot import subplots, cm
import sklearn.model_selection as skm
from ISLP import load_data, confusion_table
```

Ta cũng collect các new imports cần cho lab.

```
In [2]: from sklearn.svm import SVC
from ISLP.svm import plot as plot_svm
from sklearn.metrics import RocCurveDisplay
```

Ta sẽ dùng `RocCurveDisplay.from_estimator()` để produce several ROC plots, với shorthand `roc_curve`.

RocCurveDisplay

```
In [3]: roc_curve = RocCurveDisplay.from_estimator # shorthand
```

9.6.1 Support Vector Classifier

Ta dùng `SVC()` từ `sklearn` để fit support vector classifier cho một value cụ thể của parameter `C`. Argument `C` cho phép specify cost của margin violation. Khi cost nhỏ, margins wide và nhiều support vectors nằm trên hoặc violate margin. Khi `C` lớn, margins narrow và có ít support vectors trên hoặc violating margin.

Ta demonstrate `SVC()` trên two-dimensional example để plot resulting decision boundary. Bắt đầu bằng generating observations thuộc two classes và kiểm tra classes có linearly separable hay không.

```
In [4]: rng = np.random.default_rng(1)
X = rng.standard_normal((50, 2))
y = np.array([-1]*25+[1]*25)
X[y==1] += 1
fig, ax = subplots(figsize=(8,8))
ax.scatter(X[:,0],
           X[:,1],
           c=y,
           cmap=cm.coolwarm);
```

Chúng không linearly separable. Ta fit classifier.

```
In [5]: svm_linear = SVC(C=10, kernel='linear')
svm_linear.fit(X, y)
```

```
Out[5]: SVC(C=10, kernel='linear')
```

Support vector classifier với two features có thể visualize bằng cách plot values của decision function. Package `ISLP` có function cho việc này, inspired by a similar example trong `sklearn docs`.

decision function

```
In [6]: fig, ax = subplots(figsize=(8,8))
plot_svm(X,
         y,
         svm_linear,
         ax=ax)
```

Decision boundary là linear vì ta dùng `kernel='linear'`. Support vectors được mark bằng dấu +, remaining observations là circles. Nếu dùng smaller value của cost parameter thì sao?

```
In [7]: svm_linear_small = SVC(C=0.1, kernel='linear')
svm_linear_small.fit(X, y)
fig, ax = subplots(figsize=(8,8))
plot_svm(X,
         y,
         svm_linear_small,
         ax=ax)
```

Với smaller cost parameter, ta obtain larger number of support vectors vì margin wider. Với linear kernels, coefficients của linear decision boundary có thể extract như sau.

```
In [8]: svm_linear.coef_
```

Out[8]: `array([[1.173 , 0.7734]])`

Vì support vector machine là estimator trong sklearn, ta dùng usual machinery để tune nó.

```
In [9]: kfold = skm.KFold(5,
                        random_state=0,
                        shuffle=True)
grid = skm.GridSearchCV(svm_linear,
                        {'C': [0.001, 0.01, 0.1, 1, 5, 10, 100]},
                        refit=True,
                        cv=kfold,
                        scoring='accuracy')
grid.fit(X, y)
grid.best_params_
```

Out[9]: `{'C': 1}`

Ta có thể access cross-validation errors cho mỗi model qua `grid.cv_results_`. Vì object này print nhiều detail, ta chỉ extract accuracy results.

```
In [10]: grid.cv_results_[('mean_test_score')]
```

Out[10]: `array([0.46, 0.46, 0.72, 0.74, 0.74, 0.74, 0.74])`

$C = 1$ cho highest cross-validation accuracy 0.74, dù same accuracy cũng xuất hiện ở several values của C . Classifier `grid.best_estimator_` có thể predict class label cho test observations. Ta generate một test data set.

```
In [11]: X_test = rng.standard_normal((20, 2))
y_test = np.array([-1]*10+[1]*10)
X_test[y_test==1] += 1
```

Bây giờ predict class labels của test observations bằng best model selected by cross-validation.

```
In [12]: best_ = grid.best_estimator_
y_test_hat = best_.predict(X_test)
confusion_table(y_test_hat, y_test)
```

```
Out[12]: Truth      -1      1
Predicted
-1          8      4
1           2      6
```

Với value này của C , 70% test observations được correctly classified. Nếu dùng $C = 0.001$ thì sao?

```
In [13]: svm_ = SVC(C=0.001,
                    kernel='linear').fit(X, y)
y_test_hat = svm_.predict(X_test)
confusion_table(y_test_hat, y_test)
```

```
Out[13]: Truth      -1      1
Predicted
```

```
-1      2      0
1       8     10
```

Trong case này, 60% test observations được correctly classified.

Tiếp theo xét situation mà two classes linearly separable. Khi đó có thể tìm optimal separating hyperplane bằng `SVC()`. Ta further separate two classes trong simulated data.

```
In [14]: X[y==1] += 1.9;
fig, ax = subplots(figsize=(8,8))
ax.scatter(X[:,0], X[:,1], c=y, cmap=cm.coolwarm);
```

Bây giờ observations chỉ vừa đủ linearly separable.

```
In [15]: svm_ = SVC(C=1e5, kernel='linear').fit(X, y)
y_hat = svm_.predict(X)
confusion_table(y_hat, y)
```

```
Out[15]: Truth      -1      1
Predicted
-1         25      0
1          0     25
```

Ta fit support vector classifier và plot resulting hyperplane bằng very large C để không misclassify training observations.

```
In [16]: fig, ax = subplots(figsize=(8,8))
plot_svm(X,
         y,
         svm_,
         ax=ax)
```

Không có training errors và chỉ three support vectors được dùng. Large C cũng có nghĩa three support points này nằm trên margin và define nó. Classifier phụ thuộc vào chỉ three data points có thể không stable trên test data; ta thử smaller C .

```
In [17]: svm_ = SVC(C=0.1, kernel='linear').fit(X, y)
y_hat = svm_.predict(X)
confusion_table(y_hat, y)
```

```
Out[17]: Truth      -1      1
Predicted
-1         25      0
1          0     25
```

Với $C = 0.1$, vẫn không misclassify training observations, nhưng margin rộng hơn nhiều và dùng twelve support vectors. Chúng jointly define orientation của decision boundary; vì có nhiều points hơn nên boundary stable hơn. Model này có thể perform tốt hơn trên test data so với $C = 10^5$.

```
In [18]: fig, ax = subplots(figsize=(8,8))
plot_svm(X,
         y,
         svm_,
         ax=ax)
```

9.6.2 Support Vector Machine

Để fit SVM với non-linear kernel, ta vẫn dùng `SVC()` nhưng chọn khác parameter `kernel`. Polynomial kernel dùng `kernel="poly"`; radial kernel dùng `kernel="rbf"`. Với polynomial kernel, argument `degree` specifies d trong (9.22); với radial basis kernel, `gamma` specifies γ trong (9.24).

Ta generate data có non-linear class boundary.

```
In [19]: X = rng.standard_normal((200, 2))
X[:100] += 2
X[100:150] -= 2
y = np.array([1]*150+[2]*50)
```

Plot data cho thấy class boundary thực sự non-linear.

```
In [20]: fig, ax = subplots(figsize=(8,8))
ax.scatter(X[:,0],
           X[:,1],
           c=y,
           cmap=cm.coolwarm)
```

```
Out[20]: <matplotlib.collections.PathCollection at 0x7faa9ba52eb0>
```

Data được randomly split thành training và testing groups. Ta fit training data bằng `SVC()` với radial kernel và $\gamma = 1$.

```
In [21]: (X_train,
X_test,
y_train,
y_test) = skm.train_test_split(X,
                                y,
                                test_size=0.5,
                                random_state=0)
svm_rbf = SVC(kernel="rbf", gamma=1, C=1)
svm_rbf.fit(X_train, y_train)
```

Plot cho thấy resulting SVM có decidedly non-linear boundary.

```
In [22]: fig, ax = subplots(figsize=(8,8))
plot_svm(X_train,
         y_train,
         svm_rbf,
         ax=ax)
```

Có khá nhiều training errors trong fit này. Nếu tăng C , ta có thể reduce training errors, nhưng price là decision boundary irregular hơn và có risk overfitting.

```
In [23]: svm_rbf = SVC(kernel="rbf", gamma=1, C=1e5)
svm_rbf.fit(X_train, y_train)
fig, ax = subplots(figsize=(8,8))
plot_svm(X_train,
         y_train,
         svm_rbf,
         ax=ax)
```

Ta dùng `skm.GridSearchCV()` để select best γ và C cho radial-kernel SVM.

In [24]:

```
kfold = skm.KFold(5,
                  random_state=0,
                  shuffle=True)
grid = skm.GridSearchCV(svm_rbf,
                        {'C': [0.1, 1, 10, 100, 1000],
                        'gamma': [0.5, 1, 2, 3, 4]},
                        refit=True,
                        cv=kfold,
                        scoring='accuracy');
grid.fit(X_train, y_train)
grid.best_params_
```

Out[24]:

```
{'C': 100, 'gamma': 1}
```

Output report best parameters là $C = 100$, $\gamma = 1$. Source text ngay sau output ghi rằng best choice dưới five-fold CV là $C = 1$, $\gamma = 0.5$, và several other values cũng đạt same value; fidelity lock giữ cả output và prose đúng như nguồn.

In [25]:

```
best_svm = grid.best_estimator_
fig, ax = subplots(figsize=(8,8))
plot_svm(X_train,
         y_train,
         best_svm,
         ax=ax)
y_hat_test = best_svm.predict(X_test)
confusion_table(y_hat_test, y_test)
```

Out[25]:

Truth	1	2
Predicted		
1	69	6
2	6	19

Với các parameters này, 12% test observations bị misclassified.

9.6.3 ROC Curves

SVMs và support vector classifiers output class labels cho mỗi observation, nhưng ta cũng có thể obtain fitted values—numerical scores dùng để derive class labels. Với support vector classifier, fitted value cho $X = (X_1, \dots, X_p)^T$ có form $\hat{\beta}_0 + \hat{\beta}_1 X_1 + \dots + \hat{\beta}_p X_p$. Với non-linear-kernel SVM, equation là (9.23). Sign của fitted value determine side của decision boundary. Nếu fitted value exceeds zero thì observation được assign vào one class; nếu below zero thì vào other class. Changing threshold tạo bias toward one class hay class kia; xét range of thresholds cho ingredients của ROC plot. Values này access bằng method `decision_function()` của fitted SVM estimator.

`RocCurveDisplay.from_estimator()` (shorthand `roc_curve()`) produce ROC curve. Nó nhận fitted estimator, model matrix X và labels y . Argument `name` dùng trong legend, `color` cho line color, và results được plot trên axis object `ax`.

In [26]:

```
fig, ax = subplots(figsize=(8,8))
roc_curve(best_svm,
         X_train,
         y_train,
```

decision_function

```
name='Training',
color='r',
ax=ax);
```

Trong example này, SVM có vẻ provide accurate predictions. Tăng γ tạo more flexible fit và có thể improve training accuracy hơn nữa.

```
In [27]: svm_flex = SVC(kernel="rbf",
                    gamma=50,
                    C=1)
svm_flex.fit(X_train, y_train)
fig, ax = subplots(figsize=(8,8))
roc_curve(svm_flex,
          X_train,
          y_train,
          name='Training $\gamma=50$',
          color='r',
          ax=ax);
```

Nhưng các ROC curves trên đều từ training data. Ta quan tâm hơn prediction accuracy trên test data. Khi compute ROC curves trên test data, model với $\gamma = 0.5$ dường như provide most accurate results.

```
In [28]: roc_curve(svm_flex,
                  X_test,
                  y_test,
                  name='Test $\gamma=50$',
                  color='b',
                  ax=ax)
fig;
```

Xem tuned SVM của ta.

```
In [29]: fig, ax = subplots(figsize=(8,8))
for (X_, y_, c, name) in zip(
    (X_train, X_test),
    (y_train, y_test),
    ('r', 'b'),
    ('CV tuned on training',
     'CV tuned on test')):
    roc_curve(best_svm,
              X_,
              y_,
              name=name,
              ax=ax,
              color=c)
```

9.6.4 SVM with Multiple Classes

Nếu response là factor có more than two levels, SVC() perform multi-class classification bằng one-versus-one khi `decision_function_shape='ovo'` hoặc one-versus-rest⁴ khi `decision_function_shape='ovr'`. Ta briefly explore setting này bằng cách generate third class.

```
In [30]: rng = np.random.default_rng(123)
X = np.vstack([X, rng.standard_normal((50, 2))])
y = np.hstack([y, [0]*50])
X[y==0,1] += 2
```

⁴One-versus-rest còn gọi là one-versus-all.

```
fig, ax = subplots(figsize=(8,8))
ax.scatter(X[:,0], X[:,1], c=y, cmap=cm.coolwarm);
```

Ta fit SVM vào data.

In [31]:

```
svm_rbf_3 = SVC(kernel="rbf",
                 C=10,
                 gamma=1,
                 decision_function_shape='ovo');
svm_rbf_3.fit(X, y)
fig, ax = subplots(figsize=(8,8))
plot_svm(X,
         y,
         svm_rbf_3,
         scatter_cmap=cm.tab10,
         ax=ax)
```

SVR

Library `sklearn.svm` cũng có thể perform support vector regression với numerical response bằng estimator `SupportVectorRegression()`.

9.6.5 Application to Gene Expression Data

Ta examine data set `Khan`, gồm tissue samples tương ứng four distinct types of small round blue cell tumors. Với mỗi tissue sample có gene expression measurements. Data gồm training data `xtrain`, `ytrain` và testing data `xtest`, `ytest`.

Xem dimensions của data.

In [32]:

```
Khan = load_data('Khan')
Khan['xtrain'].shape, Khan['xtest'].shape
```

Out[32]:

```
((63, 2308), (20, 2308))
```

Data set gồm expression measurements cho 2,308 genes. Training và test sets có 63 và 20 observations. Ta dùng support vector approach để predict cancer subtype từ gene expression. Vì number of features rất lớn relative to observations, linear kernel là appropriate; additional flexibility của polynomial/radial kernel là unnecessary.

In [33]:

```
khan_linear = SVC(kernel='linear', C=10)
khan_linear.fit(Khan['xtrain'], Khan['ytrain'])
confusion_table(khan_linear.predict(Khan['xtrain']),
                Khan['ytrain'])
```

Out[33]:

Truth	1	2	3	4
Predicted				
1	8	0	0	0
2	0	23	0	0
3	0	0	12	0
4	0	0	0	20

Không có training errors. Điều này không surprising vì large number of variables relative to observations khiến việc tìm hyperplanes fully separate classes khá easy.

Ta quan tâm hơn performance trên test observations.

In [34]:

```
confusion_table(khan_linear.predict(Khan['xtest']),
                 Khan['ytest'])
```

Out[34]:

```
Truth
Predicted
1      3   0   0   0
2      0   6   2   0
3      0   0   4   0
4      0   0   0   5
```

Dùng $C = 10$ tạo two test-set errors trên data này.

9.7 Exercises

Conceptual

- Problem này liên quan đến hyperplanes trong two dimensions.
 - Sketch hyperplane $1 + 3X_1 - X_2 = 0$. Indicate set of points thỏa $1 + 3X_1 - X_2 > 0$, và set thỏa $1 + 3X_1 - X_2 < 0$.
 - Trên same plot, sketch hyperplane $-2 + X_1 + 2X_2 = 0$. Indicate set of points thỏa $-2 + X_1 + 2X_2 > 0$, và set thỏa $-2 + X_1 + 2X_2 < 0$.
- Ta đã thấy trong $p = 2$ dimensions, linear decision boundary có form $\beta_0 + \beta_1 X_1 + \beta_2 X_2 = 0$. Bây giờ investigate một non-linear decision boundary.
 - Sketch curve

$$(1 + X_1)^2 + (2 - X_2)^2 = 4.$$
 - Trên sketch, indicate set of points thỏa

$$(1 + X_1)^2 + (2 - X_2)^2 > 4,$$
 và set thỏa

$$(1 + X_1)^2 + (2 - X_2)^2 \leq 4.$$
 - Giả sử classifier assign observation vào blue class nếu $(1 + X_1)^2 + (2 - X_2)^2 > 4$, và red class otherwise. Observations $(0, 0)$, $(-1, 1)$, $(2, 2)$, $(3, 8)$ được classify vào class nào?
 - Argue rằng decision boundary ở (c) không linear theo X_1, X_2 , nhưng linear theo X_1, X_1^2, X_2, X_2^2 .
- Ta explore maximal margin classifier trên toy data set.
 - Có $n = 7$ observations trong $p = 2$ dimensions, mỗi observation có associated class label:

Obs.	X_1	X_2	Y
1	3	4	Red
2	2	2	Red
3	4	4	Red
4	1	4	Red
5	2	1	Blue
6	4	3	Blue
7	4	1	Blue

Sketch observations.

- (b) Sketch optimal separating hyperplane và provide equation của hyperplane này (form (9.1)).
- (c) Describe classification rule cho maximal margin classifier, ví dụ “Classify to Red if $\beta_0 + \beta_1 X_1 + \beta_2 X_2 > 0$, and classify to Blue otherwise.” Provide values của $\beta_0, \beta_1, \beta_2$.
- (d) Trên sketch, indicate margin cho maximal margin hyperplane.
- (e) Indicate support vectors cho maximal margin classifier.
- (f) Argue rằng slight movement của seventh observation sẽ không affect maximal margin hyperplane.
- (g) Sketch một hyperplane không phải optimal separating hyperplane, và provide equation.
- (h) Draw thêm một observation để two classes không còn separable by a hyperplane.

Applied

- 4. Generate simulated two-class data set với 100 observations và two features có visible nhưng non-linear separation giữa classes. Show rằng trong setting này, SVM với polynomial kernel (degree greater than 1) hoặc radial kernel outperform support vector classifier trên training data. Technique nào perform best trên test data? Make plots và report training/test error rates để support assertions.
- 5. Ta đã thấy SVM với non-linear kernel có thể perform classification bằng non-linear decision boundary. Bây giờ cho thấy logistic regression dùng non-linear transformations của features cũng có thể tạo non-linear decision boundary.
 - (a) Generate data set với $n = 500$, $p = 2$, observations thuộc two classes với quadratic decision boundary. Ví dụ:

```
rng = np.random.default_rng(5)
x1 = rng.uniform(size=500) - 0.5
x2 = rng.uniform(size=500) - 0.5
y = x1**2 - x2**2 > 0
```

- (b) Plot observations, colored theo class labels; display X_1 trên x-axis và X_2 trên y-axis.
- (c) Fit logistic regression model dùng X_1, X_2 làm predictors.
- (d) Apply model lên training data để obtain predicted class label cho mỗi observation. Plot observations colored theo predicted labels. Decision boundary nên linear.
- (e) Fit logistic regression model dùng non-linear functions của X_1, X_2 làm predictors (ví dụ $X_1^2, X_1 \times X_2, \log(X_2)$, v.v.).
- (f) Apply model lên training data để obtain predicted class labels và plot. Decision boundary phải obviously non-linear; nếu chưa, repeat (a)–(e) cho đến khi có example phù hợp.
- (g) Fit support vector classifier với X_1, X_2 làm predictors. Obtain

- class prediction cho mỗi training observation và plot colored theo predicted class labels.
- (h) Fit SVM dùng non-linear kernel. Obtain class prediction cho mỗi training observation và plot colored theo predicted class labels.
 - (i) Comment on results.
6. Cuối Section 9.6.1 có claim rằng khi data chỉ barely linearly separable, support vector classifier với small C misclassify vài training observations có thể perform better trên test data so với huge C không misclassify training observations. Investigate claim này.
- (a) Generate two-class data với $p = 2$ sao cho classes chỉ barely linearly separable.
 - (b) Compute cross-validation error rates cho support vector classifiers với range of C values. Với mỗi C , bao nhiêu training observations bị misclassified, và relation với cross-validation errors là gì?
 - (c) Generate appropriate test data set và compute test errors cho từng C . C nào cho fewest test errors? Compare với C values cho fewest training errors và fewest cross-validation errors.
 - (d) Discuss results.
7. Dùng support vector approaches để predict một car có high hay low gas mileage dựa trên data set **Auto**.
- (a) Create binary variable bằng 1 cho cars có gas mileage above median và 0 cho below median.
 - (b) Fit support vector classifier với various C values để predict high-/low mileage. Report cross-validation errors và comment. Cần fit classifier without gas mileage variable để results sensible.
 - (c) Repeat (b) bằng SVMs với radial và polynomial basis kernels, thử different values của **gamma**, **degree**, và C . Comment on results.
 - (d) Make plots để support assertions trong (b), (c).
Hint: Trong lab, `plot_svm()` dùng cho fitted SVMs. Khi $p > 2$, có thể dùng keyword argument **features** để create plots displaying pairs of variables at a time.
8. Problem này dùng data set **OJ** trong package **ISLP**.
- (a) Create training set gồm random sample 800 observations và test set gồm remaining observations.
 - (b) Fit support vector classifier trên training data với $C = 0.01$, response là **Purchase**, other variables là predictors. Có bao nhiêu support points?
 - (c) Training và test error rates là bao nhiêu?
 - (d) Dùng cross-validation để select optimal C . Xét values từ 0.01 đến 10.
 - (e) Compute training và test error rates với new C .
 - (f) Repeat (b)–(e) bằng SVM với radial kernel. Dùng default **gamma**.

- (g) Repeat (b)–(e) bằng SVM với polynomial kernel, set `degree=2`.
- (h) Overall, approach nào cho best results?

10

Deep Learning

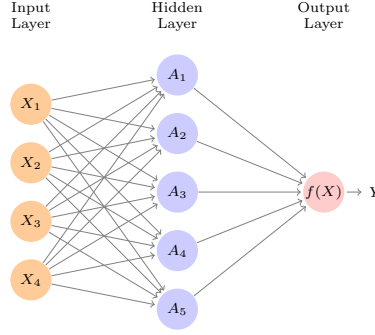
Chapter này trình bày topic quan trọng *deep learning*. Tại thời điểm sách được viết (2020), deep learning là một very active area of research trong machine learning và artificial intelligence communities. Cornerstone của deep learning là *neural network*.

deep learning;
neural network

Neural networks trở nên nổi tiếng vào late 1980s. Khi đó có nhiều excitement và một mức hype đáng kể, và approach này là động lực cho các meetings Neural Information Processing Systems (NeurIPS, trước đây NIPS) được tổ chức hằng năm, thường ở những locations đặc biệt như ski resorts. Sau đó là giai đoạn synthesis: properties của neural networks được machine learners, mathematicians và statisticians analyze; algorithms được improve và methodology dần ổn định. Rồi SVMs, boosting, và random forests xuất hiện, khiến neural networks phần nào mất ưu thế. Một lý do là neural networks cần khá nhiều tinkering, còn các newer methods automatic hơn. Trên nhiều problems, newer methods cũng outperform poorly-trained neural networks. Đây là status quo trong first decade của new millennium.

Dù vậy, một core group những neural-network enthusiasts vẫn push technology trên ever-larger computing architectures và data sets. Sau 2010, neural networks resurfaced dưới tên *deep learning*, với new architectures, additional bells and whistles, cùng nhiều success stories trong image/video classification, speech và text modeling. Nhiều researchers tin rằng main reason cho thành công này là availability của ever-larger training datasets, nhờ wide-scale digitization trong science và industry.

Trong chapter này, ta discuss basics của neural networks và deep learning, rồi đi vào specializations cho specific problems, như convolutional neural networks (CNNs) cho image classification và recurrent neural networks (RNNs) cho time series cùng other sequences. Ta cũng demonstrate các models bằng Python package `torch` cùng một số helper packages. Material trong chapter này slightly more challenging so với phần khác của sách.



HÌNH 10.1. Neural network với single hidden layer. Hidden layer computes activations $A_k = h_k(X)$ là nonlinear transformations của linear combinations của inputs $X_1, X_2, \dots, X_p$. Vì thế A_k không directly observed. Functions $h_k(\cdot)$ không fixed in advance mà được learned trong training. Output layer là linear model dùng activations A_k làm inputs, tạo function $f(X)$.

10.1 Single Layer Neural Networks

Một neural network nhận input gồm p variables $X = (X_1, X_2, \dots, X_p)$ và build nonlinear function $f(X)$ để predict response Y . Ở các chapters trước ta đã build nonlinear prediction models bằng trees, boosting và generalized additive models. Điểm khác biệt của neural networks là particular structure của model. Figure 10.1 show một feed-forward neural network để model quantitative response với $p = 4$ predictors. Trong neural-network terminology, four features $X_1, \dots, X_4$ tạo nên units trong *input layer*. Arrows cho thấy mỗi input feeds into mỗi trong K *hidden units* (ta chọn K ; ở đây $K = 5$).

feed-forward NN;
input layer; hidden
units

Neural network model có form

$$\begin{aligned} f(X) &= \beta_0 + \sum_{k=1}^K \beta_k h_k(X) \\ &= \beta_0 + \sum_{k=1}^K \beta_k g \left(w_{k0} + \sum_{j=1}^p w_{kj} X_j \right). \end{aligned} \quad (10.1)$$

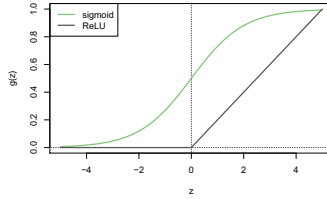
Nó được build qua two steps. First, K activations A_k , $k = 1, \dots, K$, trong hidden layer được compute từ input features:

$$A_k = h_k(X) = g \left(w_{k0} + \sum_{j=1}^p w_{kj} X_j \right). \quad (10.2)$$

activations

Ở đây $g(z)$ là nonlinear *activation function* specified in advance. Có thể xem mỗi A_k như một transformation khác nhau $h_k(X)$ của original features, tương tự basis functions ở Chapter 7. Các K activations từ hidden layer sau đó feed vào output layer:

$$f(X) = \beta_0 + \sum_{k=1}^K \beta_k A_k, \quad (10.3)$$



HÌNH 10.2. Activation functions. Piecewise-linear ReLU function popular vì efficiency và computability. Trong figure, nó được scale down một factor five để dễ compare.

đây là linear regression model theo $K = 5$ activations. Tất cả parameters $\beta_0, \dots, \beta_K$ và $w_{10}, \dots, w_{Kp}$ phải estimate từ data.

activation function

Trong early neural networks, sigmoid activation function được favored:

$$g(z) = \frac{e^z}{1 + e^z} = \frac{1}{1 + e^{-z}}, \quad (10.4)$$

chính là function dùng trong logistic regression để convert linear function thành probabilities giữa zero và one (Figure 10.2). Preferred choice trong modern neural networks là ReLU (rectified linear unit):

$$g(z) = (z)_+ = \begin{cases} 0, & z < 0, \\ z, & \text{otherwise.} \end{cases} \quad (10.5)$$

ReLU activation có thể compute và store efficiently hơn sigmoid. Dù threshold ở zero, vì nó được apply lên linear function (10.2), constant term w_{k0} shifts inflection point này.

sigmoid; ReLU;
rectified linear unit

Nói bằng lời, model trong Figure 10.1 derives five new features bằng cách compute five different linear combinations của X , rồi pass mỗi combination qua activation function $g(\cdot)$. Final model linear theo các derived variables này.

Tên *neural network* ban đầu xuất phát từ cách nghĩ hidden units analogous to neurons trong brain—với sigmoid activation, values $A_k = h_k(X)$ gần one được xem là *firing*, còn gần zero là *silent*.

Nonlinearity trong activation function $g(\cdot)$ là essential; nếu không có nó, model $f(X)$ trong (10.1) collapse thành simple linear model theo $X_1, \dots, X_p$. Nonlinear activation cũng cho phép model capture complex nonlinearities và interaction effects. Xét simple example với $p = 2$ inputs $X = (X_1, X_2)$ và $K = 2$ hidden units $h_1(X), h_2(X)$, dùng $g(z) = z^2$. Chọn

$$\begin{aligned} \beta_0 &= 0, & \beta_1 &= \frac{1}{4}, & \beta_2 &= -\frac{1}{4}, \\ w_{10} &= 0, & w_{11} &= 1, & w_{12} &= 1, \\ w_{20} &= 0, & w_{21} &= 1, & w_{22} &= -1. \end{aligned} \quad (10.6)$$

Từ (10.2),

$$\begin{aligned} h_1(X) &= (0 + X_1 + X_2)^2, \\ h_2(X) &= (0 + X_1 - X_2)^2. \end{aligned} \quad (10.7)$$

Thế (10.7) vào (10.1):

$$\begin{aligned} f(X) &= 0 + \frac{1}{4}(0 + X_1 + X_2)^2 - \frac{1}{4}(0 + X_1 - X_2)^2 \\ &= \frac{1}{4} [(X_1 + X_2)^2 - (X_1 - X_2)^2] \\ &= X_1 X_2. \end{aligned} \quad (10.8)$$

Vì vậy sum của two nonlinear transformations của linear functions có thể tạo ra interaction. Trong practice ta không dùng quadratic $g(z)$ vì khi đó luôn obtain second-degree polynomial trong original coordinates. Sigmoid và ReLU không có limitation này.

Fitting neural network requires estimating unknown parameters trong (10.1). Với quantitative response, squared-error loss thường được dùng, nên parameters được chọn để minimize

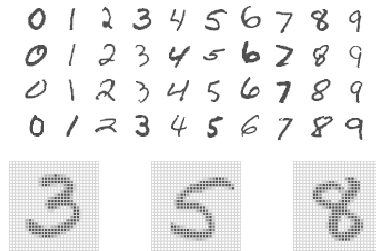
$$\sum_{i=1}^n (y_i - f(x_i))^2. \quad (10.9)$$

Details của minimization được trình bày ở Section 10.7.

10.2 Multilayer Neural Networks

Modern neural networks thường có more than one hidden layer và often many units per layer. Về theory, single hidden layer với large number of units có thể approximate most functions. Tuy nhiên learning task để discover good solution trở nên much easier với multiple layers, mỗi layer có modest size.

Ta illustrate một large dense network trên famous publicly available MNIST handwritten digit dataset.¹ Figure 10.3 cho examples. Goal là classify images vào correct digit class 0–9. Mỗi image có $p = 28 \times 28 = 784$ pixels, mỗi pixel là eight-bit grayscale value từ 0 đến 255 biểu diễn mức dark trong tiny square.



HÌNH 10.3. Examples của handwritten digits từ MNIST corpus. Mỗi grayscale image có 28×28 pixels; mỗi pixel là eight-bit number (0–255) biểu diễn mức dark. First 3, 5, và 8 được enlarge để show 784 individual pixel values.

Các pixels được store trong input vector X (ví dụ theo column order). Output là class label, represented bởi vector $Y = (Y_0, Y_1, \dots, Y_9)$ gồm 10

¹LeCun, Cortes, and Burges (2010), “The MNIST database of handwritten digits”, available at <http://yann.lecun.com/exdb/mnist>.

dummy variables, với one ở position corresponding to label và zeros elsewhere. Trong machine learning community, cách này gọi là *one-hot encoding*. Có 60,000 training images và 10,000 test images.²

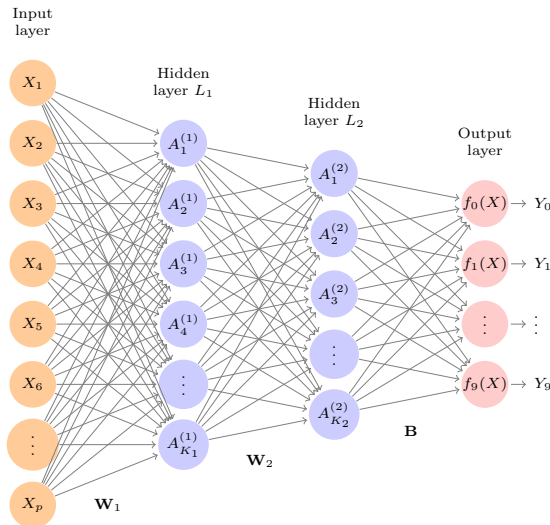
one-hot encoding

Về historical note, digit-recognition problems là catalyst thúc đẩy neural-network technology vào late 1980s ở AT&T Bell Laboratories và elsewhere. Pattern-recognition tasks kiểu này relatively simple cho humans vì visual system chiếm large fraction của brain và good recognition là evolutionary force for survival. Với machines, tasks này không simple; phải mất hơn 30 years để refine neural-network architectures đến human-level performance.

Figure 10.4 show multilayer network architecture perform tốt cho digit-classification task. Nó khác Figure 10.1 theo several ways:

- Có two hidden layers L_1 (256 units) và L_2 (128 units) thay vì one. Later ta sẽ thấy network với seven hidden layers.
- Có ten output variables thay vì one. Ở đây ten variables represent một single qualitative variable nên strongly dependent; chúng được index theo digit classes 0–9 thay vì 1–10. Tổng quát hơn, trong *multi-task learning*, một network có thể simultaneously predict different responses; tất cả responses đều influence formation của hidden layers.
- Loss function dùng để train network được tailored cho multiclass classification task.

multi-task learning



HÌNH 10.4. Neural-network diagram với two hidden layers và multiple outputs, suitable cho MNIST handwritten-digit problem. Input layer có $p = 784$ units, hidden layers $K_1 = 256$ và $K_2 = 128$ units, output layer có 10 units. Cùng intercepts (gọi là biases trong deep-learning community), network có 235,146 parameters (gọi là weights).

²Trong analog-to-digital conversion, chỉ một phần của written numeral có thể nằm trong square corresponding to a particular pixel.

First hidden layer như (10.2):

$$A_k^{(1)} = h_k^{(1)}(X) = g \left(w_{k0}^{(1)} + \sum_{j=1}^p w_{kj}^{(1)} X_j \right), \quad (10.10)$$

cho $k = 1, \dots, K_1$. Second hidden layer treats activations $A_k^{(1)}$ của first hidden layer làm inputs và compute

$$A_\ell^{(2)} = h_\ell^{(2)}(X) = g \left(w_{\ell 0}^{(2)} + \sum_{k=1}^{K_1} w_{\ell k}^{(2)} A_k^{(1)} \right), \quad (10.11)$$

cho $\ell = 1, \dots, K_2$.

Mỗi activation $A_\ell^{(2)} = h_\ell^{(2)}(X)$ là function của input vector X : dù explicitly là function của $A_k^{(1)}$ từ layer L_1 , các $A_k^{(1)}$ lại là functions của X . Với additional hidden layers cũng tương tự. Qua chain of transformations, network có thể build fairly complex transformations của X rồi feed chúng như features vào output layer.

Superscript notation như $h_\ell^{(2)}(X)$ và $w_{\ell j}^{(2)}$ trong (10.10), (10.11) cho biết activation/weight thuộc layer nào. Notation W_1 trong Figure 10.4 represent entire matrix of weights feed từ input layer tới first hidden layer L_1 . Matrix này có $785 \times 256 = 200,960$ elements; 785 chứ không 784 vì phải include intercept hay *bias* term.³

weights; bias

Mỗi element $A_k^{(1)}$ feeds vào second hidden layer L_2 qua matrix W_2 dimension $257 \times 128 = 32,896$.

Ở output layer ta có ten responses thay vì one. First compute ten different linear models tương tự (10.1):

$$\begin{aligned} Z_m &= \beta_{m0} + \sum_{\ell=1}^{K_2} \beta_{m\ell} h_\ell^{(2)}(X) \\ &= \beta_{m0} + \sum_{\ell=1}^{K_2} \beta_{m\ell} A_\ell^{(2)}, \end{aligned} \quad (10.12)$$

cho $m = 0, 1, \dots, 9$. Matrix B stores all $129 \times 10 = 1,290$ weights này.

Nếu đây là separate quantitative responses, ta chỉ cần set $f_m(X) = Z_m$. Nhưng ta muốn estimates represent class probabilities $f_m(X) = \Pr(Y = m \mid X)$ như multinomial logistic regression ở Section 4.3.5. Vì vậy dùng special *softmax* activation function (xem (4.13)):

$$f_m(X) = \Pr(Y = m \mid X) = \frac{e^{Z_m}}{\sum_{\ell=0}^9 e^{Z_\ell}}, \quad (10.13)$$

cho $m = 0, 1, \dots, 9$. Nhờ vậy 10 numbers non-negative và sum to one. Mặc dù goal là classifier, model estimate probability cho mỗi class rồi assign image vào class có highest probability.

softmax

³Cách dùng “weights” cho coefficients và “bias” cho intercepts w_{k0} trong (10.2) phổ biến trong machine learning; “bias” ở đây không nên nhầm với bias trong “bias-variance”.

Để train network với qualitative response, ta tìm coefficient estimates minimize negative multinomial log-likelihood

$$-\sum_{i=1}^n \sum_{m=0}^9 y_{im} \log(f_m(x_i)), \quad (10.14)$$

còn gọi là *cross-entropy*. Đây là generalization của criterion (4.5) cho two-class logistic regression. Details về minimization ở Section 10.7. Nếu response quantitative, ta instead minimize squared-error loss như (10.9).

cross-entropy

Table 10.1 compare test performance của neural network với two simple models từ Chapter 4 có linear decision boundaries: multinomial logistic regression và linear discriminant analysis. Improvement rất dramatic: network với dropout regularization đạt test error rate dưới 2% trên 10,000 test images. Dropout regularization được discuss ở Section 10.7.3; Section 10.9.2 trong lab trình bày code fit model này, chạy chỉ hơn two minutes trên laptop.

Method	Test Error
Neural Network + Ridge Regularization	2.3%
Neural Network + Dropout Regularization	1.8%
Multinomial Logistic Regression	7.2%
Linear Discriminant Analysis	12.7%

BẢNG 10.1. Test error rate trên MNIST data cho neural networks với two forms of regularization, cùng multinomial logistic regression và linear discriminant analysis. Trong example này, extra complexity của neural network tạo marked improvement về test error.

Cộng number of coefficients trong W_1 , W_2 và B , ta có 235,146, hơn 33 lần number $785 \times 9 = 7,065$ cần cho multinomial logistic regression. Training set có 60,000 images; dù nghe large, number of coefficients trong neural network gần four times number of training observations. Để avoid overfitting cần regularization. Example này dùng two forms: ridge regularization, similar to ridge regression Chapter 6, và dropout regularization. Cả hai được discuss ở Section 10.7.

dropout